VYBE — API Reference

VybeAudioEngine

The single audio engine instance. Accessed via `ref.read(audioEngineProvider)`. Never instantiated directly.

Lifecycle

```dart
Future<void> initialize()
```

Called once from `app.dart` post-frame. Sets up the audio session, detects device capabilities, creates the ExoPlayer pipeline, starts the stall watchdog, and restores persisted state. Safe to call multiple times — guarded by `_initialized`.

```dart
Future<void> dispose()
```

Cancels the stall watchdog, closes the state stream, and disposes the player. Called automatically when the provider scope is destroyed.

Playback

```dart
Future<void> loadQueue(List<Track> tracks, {int startIndex = 0})
```

Replaces the current queue and starts playback from `startIndex`. Skips tracks with invalid URIs silently. Waits for `JustAudioBackground` to be ready before playing.

```dart
Future<void> playTrack(Track track)
```

If the track is already in the queue, seeks to it. Otherwise calls `loadQueue([track])`.

```dart
```

```
Future<void> play()
Future<void> pause()
Future<void> togglePlayPause()
Future<void> seek(Duration position)
Future<void> skipNext()
Future<void> skipPrevious()
```

`skipPrevious()` restarts the current track if position > 3 s, otherwise seeks to the previous item.

```dart
Future<void> toggleShuffle()
Future<void> cycleLoopMode()   // off → all → one → off
```

Both persist their new state to `SharedPreferences` immediately.

```dart
Future<void> setVolume(double v)   // 0.0 – 1.0
```

Sets the ExoPlayer software gain and syncs the loudness enhancer state.

Queue

```dart
Future<void> addToQueue(Track track)   // appends to end
Future<void> addNext(Track track)      // inserts after current
Future<void> removeFromQueue(int index)
```

DSP

```dart
Future<void> setDspMode(DspMode mode)
```

Fades volume to 0, swaps effects, fades back. Total duration ~240 ms. Persists the choice.

```dart
DspMode get currentDspMode
```

```dart
```

```dart
Future<AndroidEqualizerParameters?> getEqualizerParameters()
```

Returns the live EQ parameters including band count, center frequencies, and min/max gain range. Returns `null` if the effect is not ready or mode is Transparent.

```dart
Future<void> setEqualizerBandGain(int bandIndex, double gainDb)
```

Sets a single band. Guarded — does nothing in Transparent mode.

```dart
Future<void> applyEqGains(List<double> gains)
```

Sets all 10 bands at once. Guarded — does nothing in Transparent mode.

State stream

```dart
Stream<VybePlayerState> get stateStream
VybePlayerState get currentState
```

`VybePlayerState` fields:

Field	Type	Description
`currentTrack`	`Track?`	Track currently loaded
`playerState`	`PlayerState`	ExoPlayer playing + processingState
`position`	`Duration`	Current playback position
`duration`	`Duration?`	Duration of current track
`volume`	`double`	Software volume (0.0–1.0)
`shuffleEnabled`	`bool`	
`loopMode`	`LoopMode`	`off` / `all` / `one`
`activeTier`	`PlaybackTier`	`standard` / `hiRes` / `bitPerfect`
`currentIndex`	`int`	Index in queue
`queue`	`List<Track>`	Unmodifiable snapshot

Derived getters:

```dart
bool get isPlaying     // playerState.playing
bool get isLoading     // loading or buffering
double get progress    // 0.0 - 1.0
```

Providers

All providers are defined in `lib/core/providers.dart`.

Core

Provider	Type	Description
`audioEngineProvider`	`Provider<VybeAudioEngine>`	Singleton engine
`playerStateProvider`	`StreamProvider<VybePlayerState>`	Engine state stream
`dynamicPaletteProvider`	`AsyncNotifierProvider<DynamicPalette>`	Album art colours

Player state (derived from playerStateProvider)

Provider	Type
`currentTrackProvider`	`Provider<Track?>`
`isPlayingProvider`	`Provider<bool>`
`playerPositionProvider`	`Provider<Duration>`
`playerDurationProvider`	`Provider<Duration?>`
`playbackProgressProvider`	`Provider<double>`
`queueProvider`	`Provider<List<Track>>`
`activeTierProvider`	`Provider<PlaybackTier>`

Library

Provider	Type	Description
`localTracksProvider`	`FutureProvider<List<SongModel>>`	All songs, re-runs on permission grant
`localAlbumsProvider`	`FutureProvider<List<AlbumModel>>`	All albums
`localArtistsProvider`	`FutureProvider<List<ArtistModel>>`	All artists
`filteredSongsProvider`	`Provider<AsyncValue<List<SongModel>>>`	Songs filtered by search query
`permissionGrantedProvider`	`StateProvider<bool>`	Flipped by bootstrap after permission grant

Playlists

Provider	Type	Description
`playlistRepoProvider`	`Provider<PlaylistRepository>`	Hive repository

Provider	Type	Description
`playlistsProvider`	`StreamProvider<List<VybePlaylist>>`	Reactive list from Hive listenable

Lyrics

Provider	Type	Description
`lyricsRepoProvider`	`Provider<LyricsRepository>`	Wired to API key providers
`lyricsProvider`	`FutureProvider.family<Lyrics, Track>`	Fetch for a specific track

UI state

Provider	Type	Default
`playerExpandedProvider`	`StateProvider<bool>`	`false`
`bottomNavIndexProvider`	`StateProvider<int>`	`0`
`dspModeProvider`	`StateProvider<DspMode>`	`DspMode.transparent`
`equalizerEnabledProvider`	`StateProvider<bool>`	`true`

API keys

Provider	Type
`geniusApiKeyProvider`	`StateProvider<String>`
`anthropicApiKeyProvider`	`StateProvider<String>`

Data Models

Track

```dart
```

```
class Track extends HiveObject {
  final String  id;           // 'local_{songId}'
  final String  title;
  final String  artist;
  final String  album;
  final String? albumArtUri;  // null for local files
  final String? localPath;    // content URI from MediaStore
  final int     durationMs;
  final int?    sampleRate;   // populated for Hi-Res files
  final int?    bitDepth;
  final String? codec;        // 'FLAC', 'MP3', etc.
  final String  sourceType;   // always 'local' in Phase 1

  bool get isLocal  // always true
  bool get isHiRes  // sampleRate > 48000 || bitDepth > 16
  Duration get duration
  String get qualityBadge  // 'FLAC', 'Hi-Res', or ''
}
```

VybePlaylist

```dart
class VybePlaylist extends HiveObject {
  final String   id;          // uuid v4
        String   name;
        List<String> trackIds; // ordered 'local_{songId}' list
  final DateTime createdAt;
        DateTime updatedAt;
        String?  coverArtUri;

  int get trackCount
}
```

DynamicPalette

```dart
class DynamicPalette {
  final Color tint;     // vibrant at 25% HSL lightness — background overlay
  final Color vibrant;  // brightest saturated colour — glow, progress bar, border

  static const fallback  // VYBE pink #FF1B6B
}
```

Lyrics / LyricsLine

```dart
class Lyrics {
  final List<LyricsLine> lines;
  final LyricsSource source;   // lrclib, genius, aiGenerated
  final bool isSynced;         // true = has timestamps for karaoke
  final bool isAiGenerated;
  bool get isEmpty
}

class LyricsLine {
  final String    text;
  final Duration? timestamp;  // null for unsynced plain lyrics
}

enum LyricsSource { lrclib, genius, aiGenerated }
```

PlaylistRepository

```dart
static Future<void> openBox()   // call once in main.dart

List<VybePlaylist> getAll()     // sorted by updatedAt descending
ValueListenable<Box<VybePlaylist>> listenable()  // for StreamProvider

Future<VybePlaylist> create(String name)
Future<void> rename(String id, String newName)
Future<void> delete(String id)
Future<void> addTrack(String playlistId, String trackId)
Future<void> removeTrack(String playlistId, String trackId)
Future<void> reorderTrack(String playlistId, int oldIndex, int newIndex)
Future<void> setCoverArt(String playlistId, String? artUri)
```

LyricsRepository

```dart
String? geniusApiKey;     // set by lyricsRepoProvider from provider
String? anthropicApiKey;  // set by lyricsRepoProvider from provider

Future<Lyrics> fetchLyrics(Track track)
```

The method cleans the track title and artist before querying (strips `(Official Video)`, `feat.`, `VEVO`, etc.) to maximise hit rate.

Platform Channels

HiResAudioChannel

Channel: `com.vybe.app/hi_res_audio`

```dart
static Future<DeviceAudioCapabilities> getDeviceCapabilities()
static Future<int> getRecommendedTier()   // 0=standard, 1=hiRes, 2=bitPerfect
```

`DeviceAudioCapabilities`:

```dart
int  nativeSampleRate     // e.g. 48000, 96000
int  framesPerBuffer
bool hiResSupported
bool bitPerfectSupported  // Android 14+ only
int  apiLevel
int  recommendedTier
```

AudioEffectsChannel

Channel: `com.vybe.app/audio_effects`

```dart
static Future<void> setBassBoostEnabled(bool enabled)
static Future<void> setBassBoostStrength(double strength)  // 0.0-1.0
static Future<void> setVirtualizerEnabled(bool enabled)
static Future<void> setVirtualizerStrength(double strength)
static Future<void> setLoudnessEnabled(bool enabled)
static Future<void> setLoudnessGain(double gainDb)
static Future<void> disableAll()
static Future<Map<String, dynamic>> getEffectsState()
```

> ⚠️ These effects operate at the Android system mixer level, outside the ExoPlayer pipeline. They are **not** bounded by software volume. Use `AndroidEqualizer` and `AndroidLoudnessEnhancer` (in the pipeline) for safe in-app DSP.

BitPerfectChannel

Channel: `com.vybe.app/bit_perfect`

```dart
static Future<bool> isSupported()
static Future<bool> enable()
static Future<bool> disable()
static Future<UsbDacInfo?> getUsbDacInfo()
static Future<int> getNativeSampleRate()
```

VybePersistence

```dart
static Future<VybePersistence> load()

Future<void> saveLoopMode(LoopMode mode)
Future<void> saveShuffle(bool enabled)
Future<void> saveDspMode(DspMode mode)
Future<void> savePlaybackState(bool playing)

LoopMode get loopMode
bool     get shuffle
DspMode  get dspMode
```

WaveformBars

```dart
WaveformBars({
  required bool  isPlaying,
  Color   color      = VybeColors.vybeStart,
  double  height     = 24,
  int     barCount   = 5,
  double  barWidth   = 3,
  double  barSpacing = 2,
})
```

Animated bar visualiser. Bars animate independently with staggered timing when `isPlaying` is true. Settles gracefully to 30% height when paused.

GlassCard

```dart
GlassCard({
  required Widget child,
  double?  width,
  double?  height,
  EdgeInsetsGeometry padding       = const EdgeInsets.all(16),
  EdgeInsetsGeometry margin        = EdgeInsets.zero,
  BorderRadius       borderRadius  = BorderRadius.circular(20),
  double  blurStrength             = 20,
  Color   backgroundColor          = VybeColors.surfaceGlass,
  Color   borderColor              = VybeColors.border,
  double  borderWidth              = 0.5,
  VoidCallback? onTap,
  Gradient? gradient,
})
```

`BackdropFilter` glassmorphic card. Pass `gradient` to override the solid background colour.